JECRC UNIVERSITY
BUILD YOUR WORLD

Department of Engineering

Session 2026-27

PROMPT ENGINEERING

(Course Code: BCO610A)

HW-07: FOUR PROBLEMS, HONEST LOGS

AI as a Hint, Not an Answer

Submitted By:	**Submitted To:**
Name – Ankit Kumar Giri	Mr. Ajay Sharma
Enrollment No. – 25BCON1276	Prompt Engineering
Branch – B.Tech. CSE (AI&DS)	
Semester – 3	

1. HW-07 — Objective and Requirements

HW-07 is based on the Session 7 theme “AI as a Hint, Not an Answer”. The purpose is to solve four new Easy interview problems independently first, then use AI only in a controlled way for hints and later review.

Requirement	Content to Submit
Four new Easy problems	Four student-written solutions.
Independent attempt	An initial attempt before AI assistance.
Hint logs	Timestamp, what I tried, hint received, and what I did next.
Complexity analysis	Time and space complexity based on the actual code.
AI review	AI review of each working solution without rewriting it.
Disagreement resolution	Verification of any AI claim that conflicts with my own analysis.
Hardest problem	My first wrong approach and why it failed.
Reflection	My own reflection about where I get stuck and what I learned.
Git	One commit per problem.

2. Rules I Will Follow

1. Spend the first 15 minutes attempting the problem before relying on AI hints.
2. Ask for hints, not solutions or code.
3. Log every hint and what I tried before asking.
4. Submit only code I typed and can explain.
5. Use AI for review only after my solution works.

3. Four Problems Selected for HW-07

Problem 1 — Contains Duplicate

Problem: Given an integer array, return true if any value appears at least twice; otherwise return false.

Test cases:

- [1,2,3,1] → true
- [1,2,3,4] → false
- [1,1] → true

Problem 2 — Valid Anagram

Problem: Given two strings, return true if the second string is an anagram of the first; otherwise return false.

Test cases:

- "anagram", "nagaram" → true
- "rat", "car" → false
- "a", "a" → true

Problem 3 — Valid Palindrome

Problem: Given a string, determine whether it is a palindrome after converting letters to lowercase and ignoring non-alphanumeric characters.
Test cases:

- "A man, a plan, a canal: Panama" → true
- "race a car" → false
- " " → true

Problem 4 — Maximum Subarray

Problem: Given an integer array, find the contiguous subarray with the largest sum and return that sum.
Test cases:

- [-2,1,-3,4,-1,2,1,-5,4] → 6
- [1] → 1
- [5,4,-1,7,8] → 23

Problem 1 — Contains Duplicate

Given an integer array, return true if any value appears at least twice; otherwise return false.

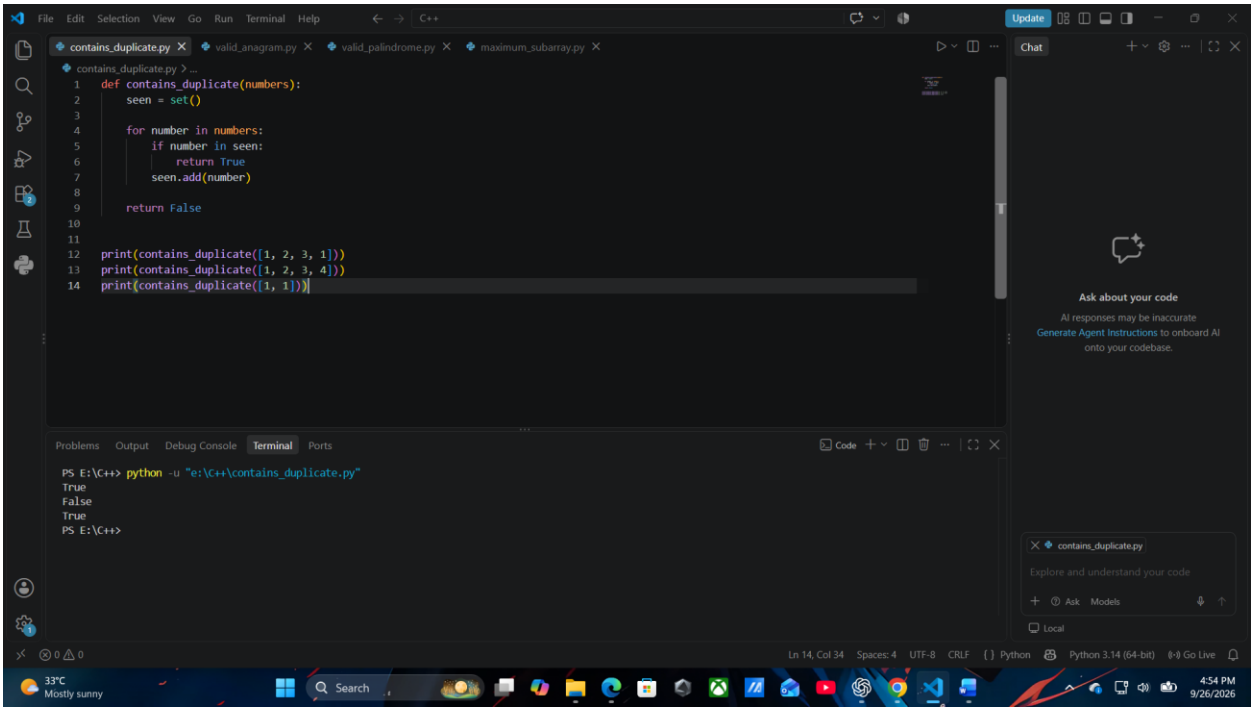
3.1 Define Input and Output

Item	Write before coding
Input	An integer array.
Output	True if a value appears more than once, otherwise False.
Edge case	[1, 1] contains duplicate values.

3.2 Hint Log

Timestamp	Level	What I tried before asking	What the hint said	What I did next
--	No AI hint	I solved the problem independently	No hint used	Tested the solution on all three test cases.

3.3 Final Student-Written Solution



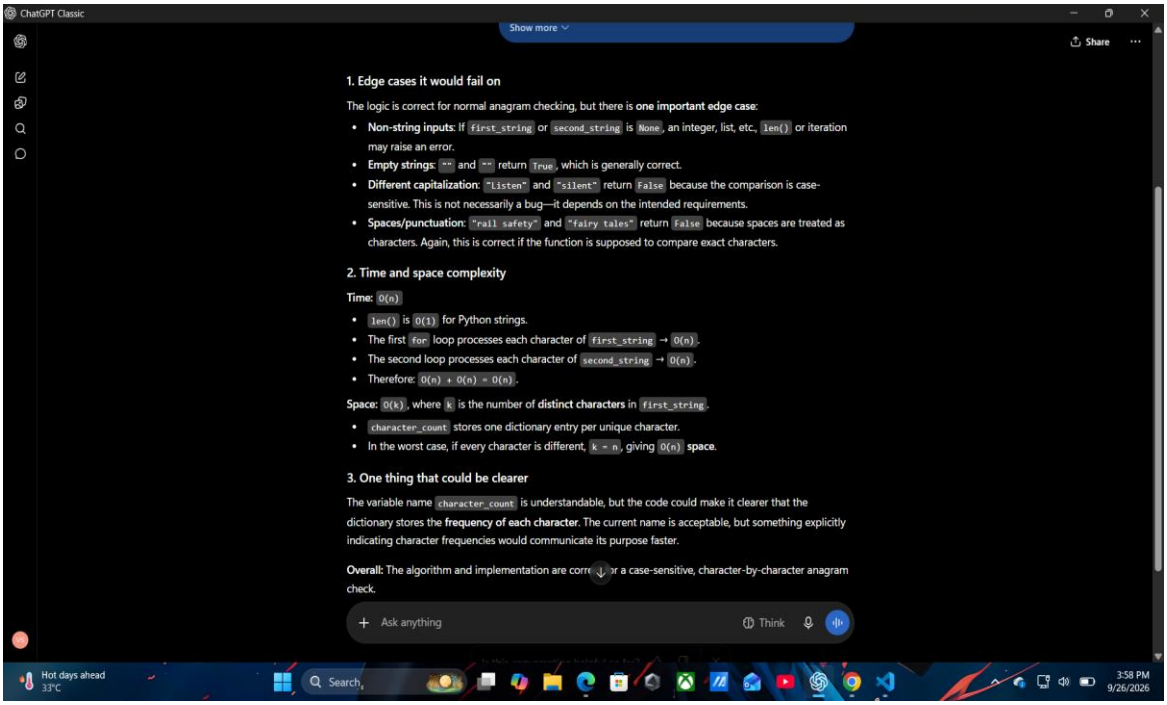
3.4 Test Results

Test	Expected	Actual	Result
A	true	True	Pass
B	false	False	Pass
C	true	True	Pass

3.5 My Complexity Claim Before AI Review

Type	My claim	Why, from my code
Time	[O(n)]	The loop checks each element once, so time is O(n).
Space	[O(n)]	The set may store up to n elements, so extra space is O(n).

3.6 AI Review


ChatGPT Classic
Show more
Share
1. Edge cases it would fail on
The logic is correct for normal anagram checking, but there is one important edge case:
Non-string inputs: If first_string or second_string is None, an integer, list, etc., len() or iteration may raise an error.
Empty strings: "" and "" return True, which is generally correct.
Different capitalization: "Listen" and "silent" return False because the comparison is case-sensitive. This is not necessarily a bug—it depends on the intended requirements.
Spaces/punctuation: "rail safety" and "fairy tales" return False because spaces are treated as characters. Again, this is correct if the function is supposed to compare exact characters.
2. Time and space complexity
Time: O(n)
len() is O(1) for Python strings.
The first for loop processes each character of first_string → O(n).
The second loop processes each character of second_string → O(n).
Therefore: O(n) + O(n) = O(n).
Space: O(k), where k is the number of distinct characters in first_string.
character_count stores one dictionary entry per unique character.
In the worst case, if every character is different, k = n, giving O(n) space.
3. One thing that could be clearer
The variable name character_count is understandable, but the code could make it clearer that the dictionary stores the frequency of each character. The current name is acceptable, but something explicitly indicating character frequencies would communicate its purpose faster.
Overall: The algorithm and implementation are corr or a case-sensitive, character-by-character anagram check.
Ask anything
Think
Hot days ahead
33°C
Search
3:58 PM
9/26/2026

Problem 2 — Valid Anagram

Given two strings, return true if the second string is an anagram of the first; otherwise return false.

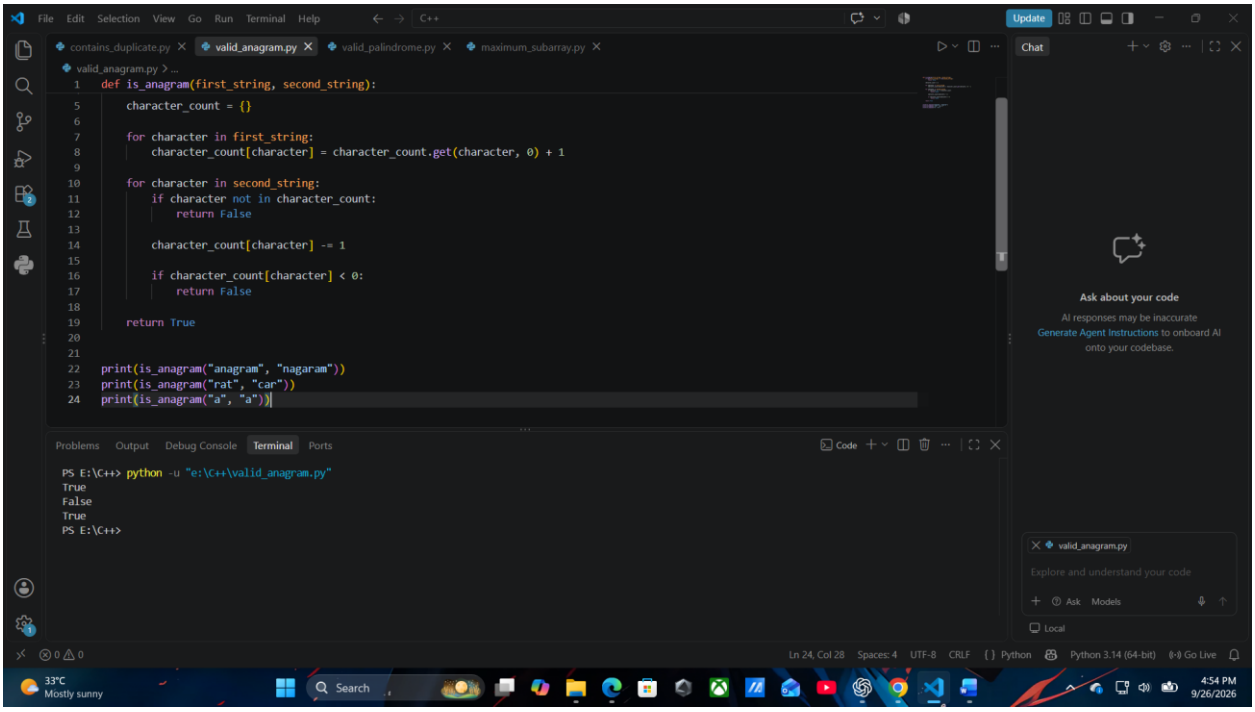
3.1 Define Input and Output

Item	Write before coding
Input	Two strings.
Output	True if both strings are anagrams; otherwise False.
Edge case	Two identical single-character strings, such as "a" and "a"

3.2 Hint Log

Timestamp	Level	What I tried before asking	What the hint said	What I did next
--	No AI hint	I solved the problem independently.	No hint used.	Tested the solution on all three test cases.

3.3 Final Student-Written Solution



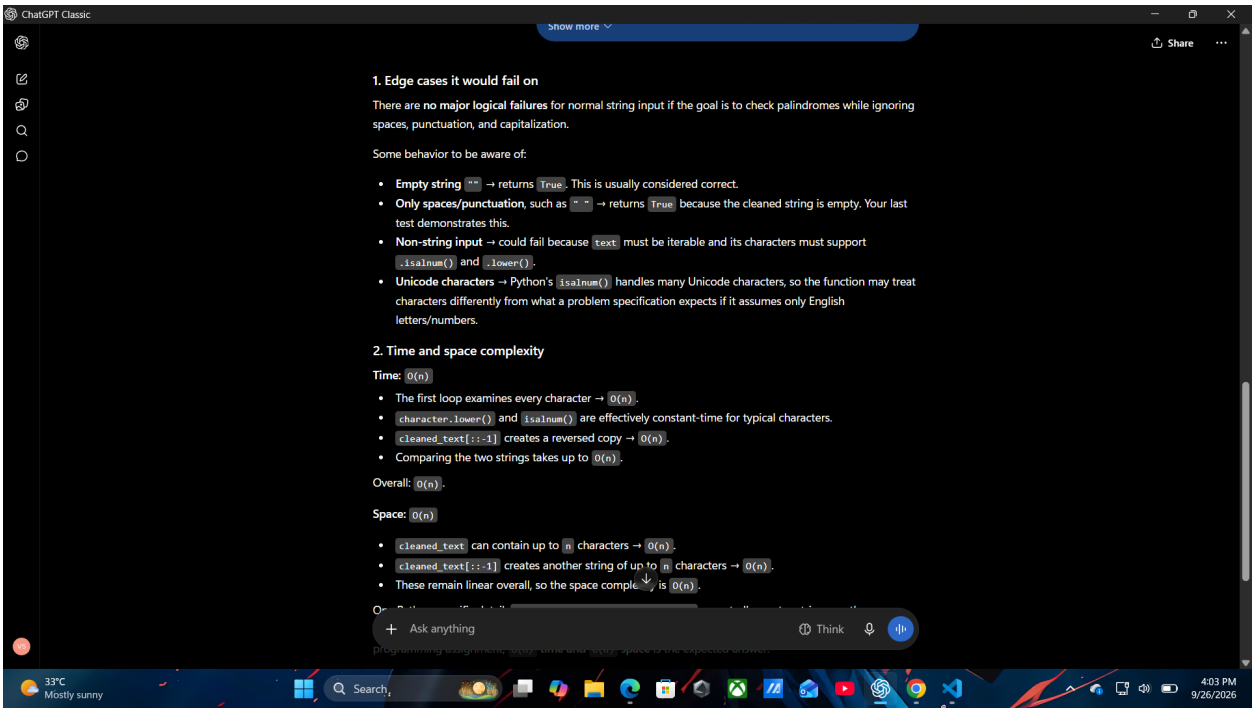
3.4 Test Results

Test	Expected	Actual	Result
A	true	True	Pass
B	false	False	Pass
C	true	True	Pass

3.5 My Complexity Claim Before AI Review

Type	My claim	Why, from my code
Time	[O(n)]	The strings are scanned once, so time is O(n).
Space	[O(n)]	The dictionary can store character counts, so extra space is O(n).

3.6 AI Review



Problem 3 — Valid Palindrome

Given a string, determine whether it is a palindrome after converting letters to lowercase and ignoring non-alphanumeric characters.

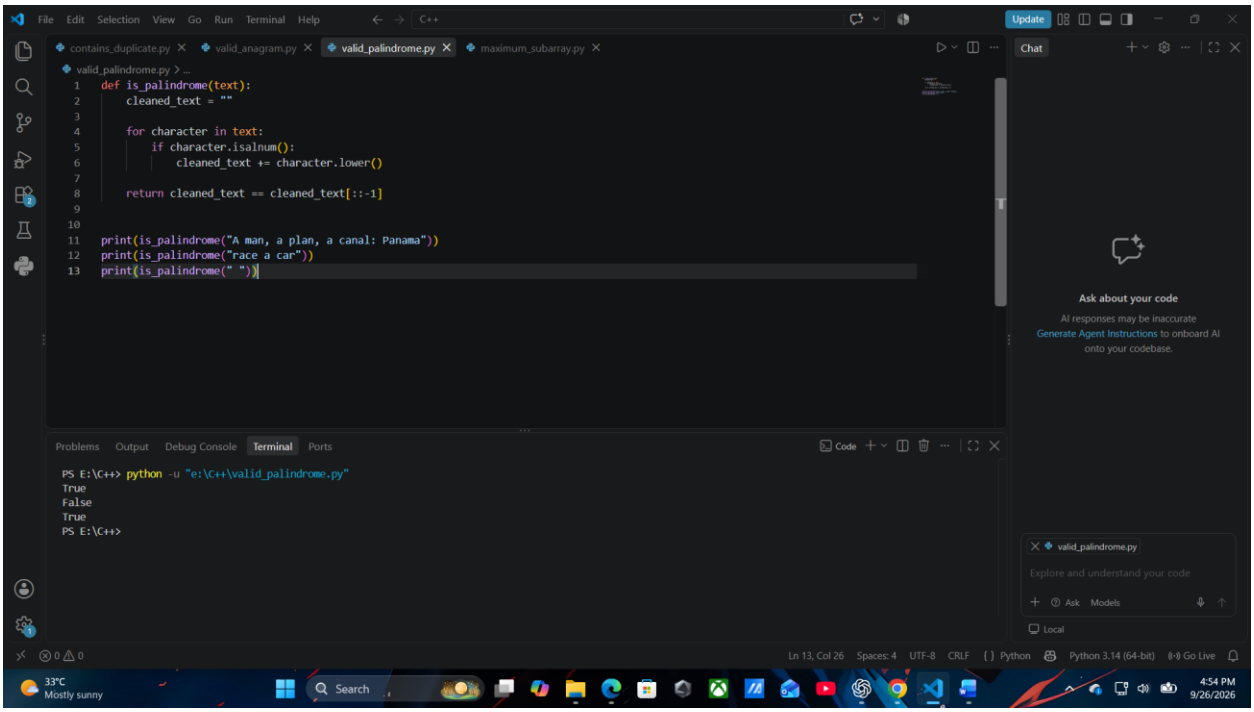
3.1 Define Input and Output

Item	Write before coding
Input	A string.
Output	True if the string is a palindrome after ignoring non-alphanumeric characters and case; otherwise False.
Edge case	An empty/space-only string.

3.2 Hint Log

Timestamp	Level	What I tried before asking	What the hint said	What I did next
--	No AI hint	I solved the problem independently.	No hint used.	Tested the solution on all three test cases.

3.3 Final Student-Written Solution



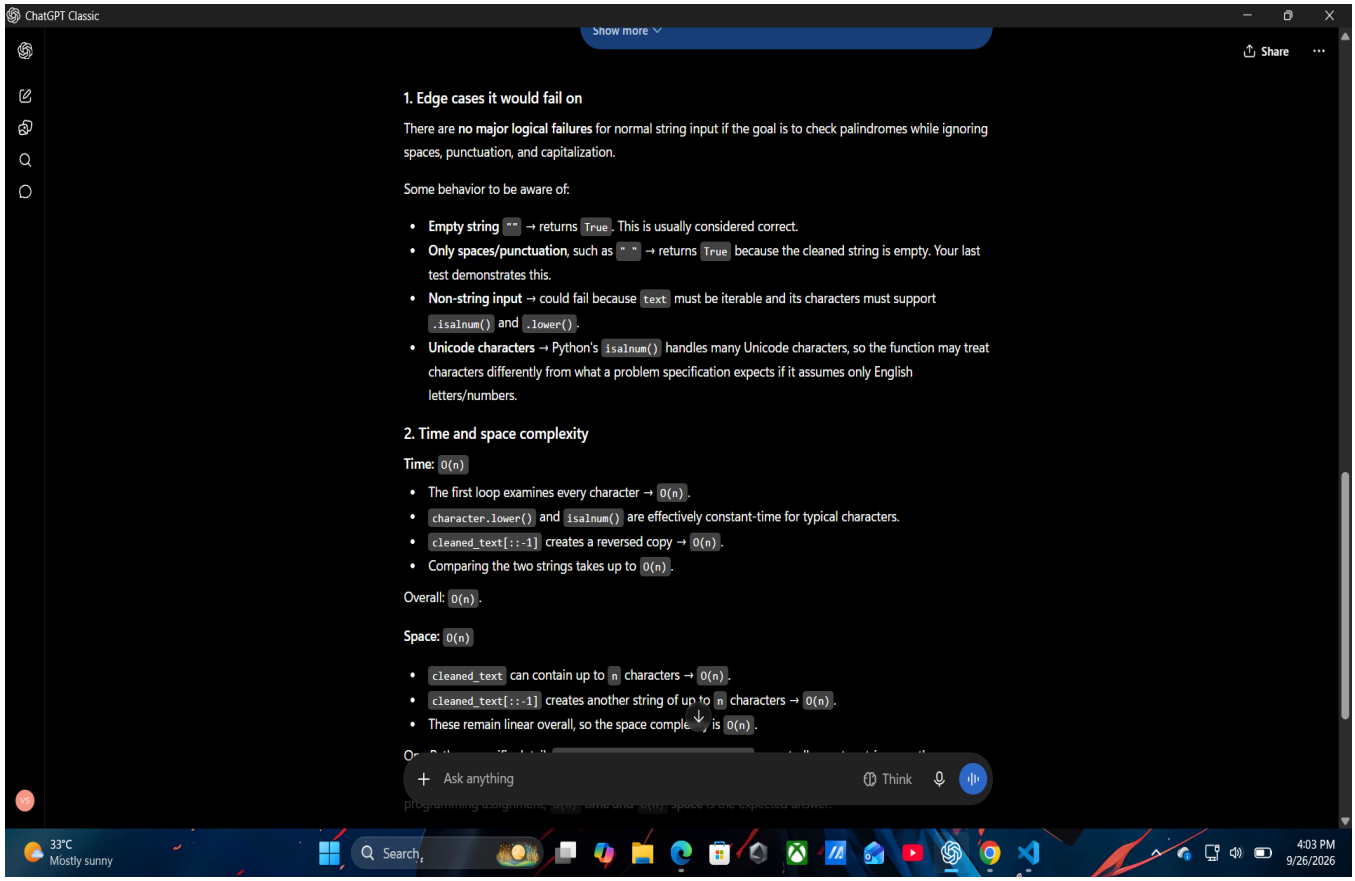
3.4 Test Results

Test	Expected	Actual	Result
A	true	True	Pass
B	false	False	Pass
C	true	True	Pass

3.5 My Complexity Claim Before AI Review

Type	My claim	Why, from my code
Time	[O(n)]	The string is scanned once, so the time complexity is O(n)
Space	[O(n)]	A cleaned string is created, requiring O(n) extra space.

3.6 AI Review


ChatGPT Classic
Show more
Share
1. Edge cases it would fail on
There are no major logical failures for normal string input if the goal is to check palindromes while ignoring spaces, punctuation, and capitalization.
Some behavior to be aware of:
Empty string "" → returns True. This is usually considered correct.
Only spaces/punctuation, such as " " → returns True because the cleaned string is empty. Your last test demonstrates this.
Non-string input → could fail because text must be iterable and its characters must support .isalnum() and .lower().
Unicode characters → Python's isalnum() handles many Unicode characters, so the function may treat characters differently from what a problem specification expects if it assumes only English letters/numbers.
2. Time and space complexity
Time: O(n)
The first loop examines every character → O(n).
character.lower() and isalnum() are effectively constant-time for typical characters.
cleaned_text[::-1] creates a reversed copy → O(n).
Comparing the two strings takes up to O(n).
Overall: O(n).
Space: O(n)
cleaned_text can contain up to n characters → O(n).
cleaned_text[::-1] creates another string of up to n characters → O(n).
These remain linear overall, so the space comple is O(n).
Ask anything
Think
33°C
Mostly sunny
Search
4:03 PM
9/26/2026

Problem 4 — Maximum Subarray

Given an integer array, find the contiguous subarray with the largest sum and return that sum.

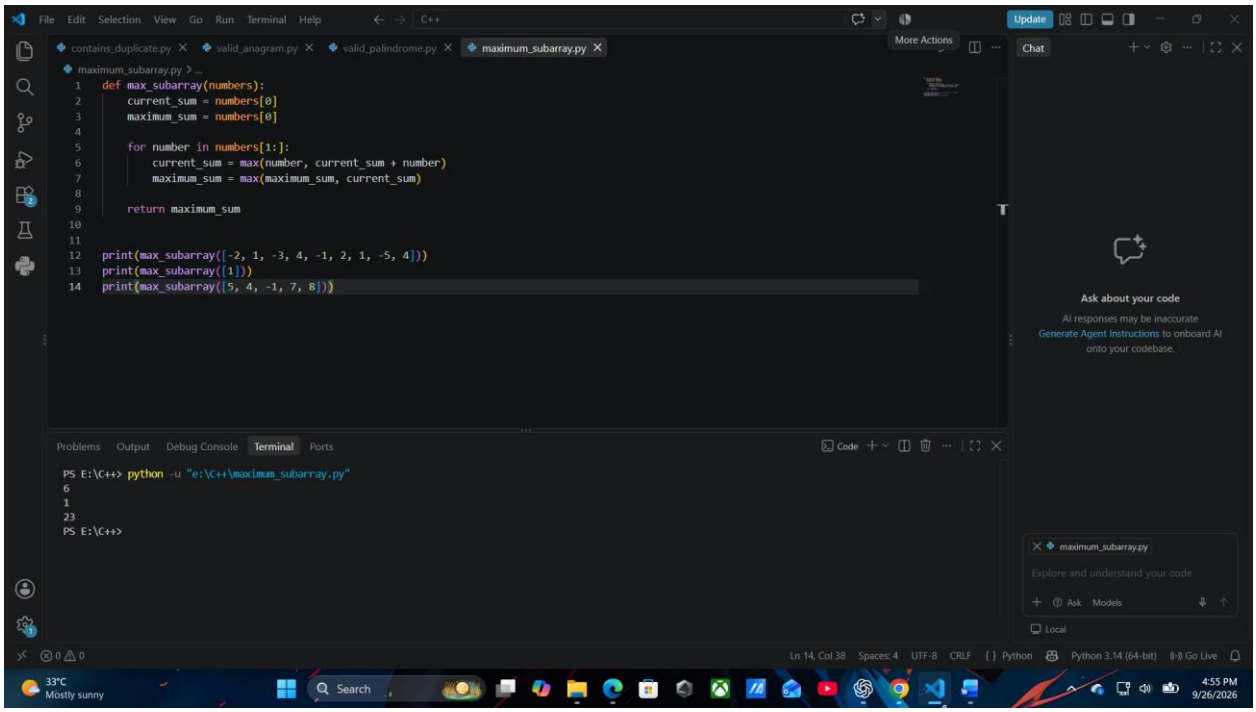
3.1 Define Input and Output

Item	Write before coding
Input	An integer array.
Output	The maximum sum of any contiguous subarray.
Edge case	An array containing only one element, such as [1].

3.2 Hint Log

Timestamp	Level	What I tried before asking	What the hint said	What I did next
--	No AI hint	I solved the problem independently.	No hint used.	Tested the solution on all three test cases.

3.3 Final Student-Written Solution



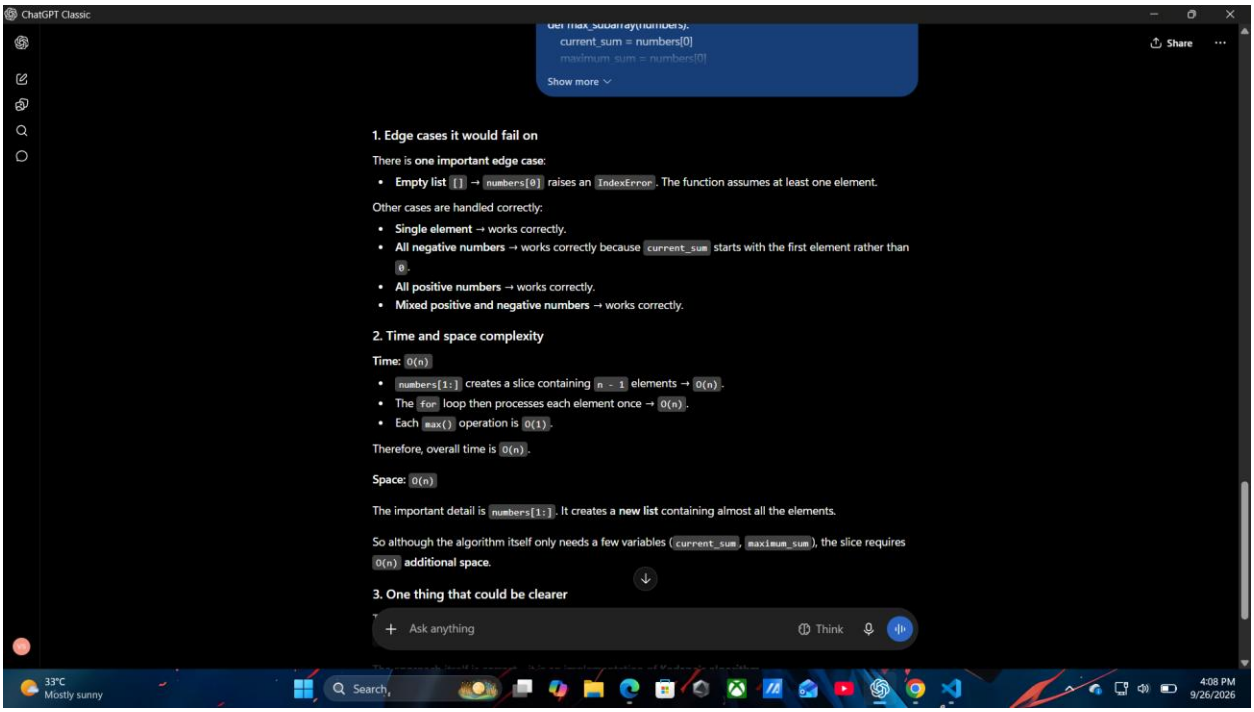
3.4 Test Results

Test	Expected	Actual	Result
A	6	6	Pass
B	1	1	Pass
C	23	23	Pass

3.5 My Complexity Claim Before AI Review

Type	My claim	Why, from my code
Time	[O(n)]	The array is scanned once, so the time complexity is O(n)
Space	[O(1)]	Only a few variables are used, so extra space is O(1).

3.6 AI Review



8. Hardest Problem — First Wrong Approach

Choose the problem that genuinely caused the most difficulty.

Point	My actual experience
Problem name	Maximum Subarray
First wrong approach	I first tried checking different contiguous subarrays and their sums.
Why I thought it would work	I thought comparing all possible sums would give the maximum.
What failed	It would require checking many subarrays and could be slow.
What I changed	I decided to keep the best sum ending at the current position.
Final approach	I scan the array once using the current sum and maximum sum.

9. Complexity Disagreement Resolution

Problem	AI claim	My analysis	Evidence	Final conclusion
Problem 1	No disagreement — O(n), O(n)	O(n), O(n)	One loop + set	Matched
Problem 2	No disagreement — O(n), O(n)	O(n), O(n)	Two scans + dictionary	Matched
Problem 3	No disagreement — O(n), O(n)	O(n), O(n)	One scan + cleaned string	Matched
Problem 4	No disagreement — O(n), O(1)	O(n), O(1)	One loop + two variables	Matched

10. Complexity Summary

Problem	Time	Space	Justification
Contains Duplicate	O(n)	O(n)	One loop + set
Valid Anagram	O(n)	O(n)	Two scans + dictionary
Valid Palindrome	O(n)	O(n)	One scan + cleaned string
Maximum Subarray	O(n)	O(1)	One loop + two variables

11. Personal Reflection

I learned that I should try solving a problem myself before using AI. AI hints can help me when I get stuck, but I should still understand and verify the solution. I also learned how to check time and space complexity from my own code. In future, I will test my code with different cases and review AI suggestions before accepting them.

12. Git Commit Order

Order	Commit / Evidence
1	Problem 1 solution committed.
2	Problem 2 solution committed.
3	Problem 3 solution committed.
4	Problem 4 solution committed.